

// Exercise 2: Implementing the Factory Method Pattern

```
interface Document {
    void open();
}

class WordDocument implements Document {
    public void open() {
        System.out.println("Opening Word Document");
    }
}

class PdfDocument implements Document {
    public void open() {
        System.out.println("Opening PDF Document");
    }
}

class ExcelDocument implements Document {
    public void open() {
        System.out.println("Opening Excel Document");
    }
}

abstract class DocumentFactory {
    public abstract Document createDocument();
}

class WordFactory extends DocumentFactory {
    public Document createDocument() {
        return new WordDocument();
    }
}

class PdfFactory extends DocumentFactory {
    public Document createDocument() {
        return new PdfDocument();
    }
}

class ExcelFactory extends DocumentFactory {
    public Document createDocument() {
        return new ExcelDocument();
    }
}

public class FactoryMethod {
    public static void main(String[] args) {
```

```

    DocumentFactory wordFactory = new WordFactory();
    Document wordDoc = wordFactory.createDocument();
    wordDoc.open();
    DocumentFactory pdfFactory = new PdfFactory();
    Document pdfDoc = pdfFactory.createDocument();
    pdfDoc.open();
    DocumentFactory excelFactory = new ExcelFactory();
    Document excelDoc = excelFactory.createDocument();
    excelDoc.open();
}
}

```

Output:

The screenshot shows the IntelliJ IDEA IDE with the 'FactoryMethod.java' file open. The code defines an abstract class 'DocumentFactory' with a 'createDocument()' method. Two concrete classes, 'WordFactory' and 'ExcelFactory', extend this abstract class. The 'WordFactory' class has a 'createDocument()' method that returns a new 'WordDocument' object. The 'ExcelFactory' class has a 'createDocument()' method that returns a new 'ExcelDocument' object. The 'FactoryMethod.java' file also contains the main logic to create and open documents using these factories.

```

10 }
11
12 class PdfDocument implements Document { 1 usage
13     public void open() { 3 usages
14         System.out.println("Opening PDF Document");
15     }
16 }
17
18 class ExcelDocument implements Document { 1 usage
19     public void open() { 3 usages
20         System.out.println("Opening Excel Document");
21     }
22 }
23
24 abstract class DocumentFactory { 6 usages 3 inheritors
25     public abstract Document createDocument(); 3 usages 3 implementations
26 }
27
28 class WordFactory extends DocumentFactory { 1 usage
29     public Document createDocument() { 3 usages
30         return new WordDocument();
31     }
32 }

```

The 'Run' tab shows the output of the program:

```

C:\Users\Naveen\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=57341" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
Opening Word Document
Opening PDF Document
Opening Excel Document
Process finished with exit code 0

```